Codebase Professionalization Playbook & Prompt Pack

Beere Kesava & Brothers Silks ERP

What this is: the architecture principles behind the refactor, a phase-by-phase roadmap with a clear definition of done, and a set of **copy-paste-ready AI prompts** — one per phase — you can hand to Claude Code (or any coding agent) to actually execute the work.

How to use it: work top to bottom. Run one prompt, review the diff, commit, then run the next. Never run two phases at once — each phase is designed to leave the app in a working state.

Companion docs: 'Backend Architecture Design' (data model + API contract) and 'Frontend & Backend Project Structure' (folder trees).

Part 1 — Architecture Principles

These are the specific lessons this codebase teaches. Each one explains *why* a refactor step exists, so the work doesn't degrade into cosmetic file-shuffling.

1. Context-as-database is the root problem — not file size

`PaymentsPage.tsx` is ~4300 lines and `WeaversPage.tsx` ~2400 lines not because of sloppiness, but because React Context was asked to be a database, a service layer, and a UI simultaneously. Splitting files without first separating **state ownership** just spreads the same coupling across more files.

Rule: decide which layer owns the data before touching folder structure. API layer first, file-splitting second.

2. A rule enforced only in the UI is not enforced

The QC pay-deduction rule (defective → zero payable; semi → deduction capped at making charge) and the jari conversion (230g per reel, 4 reels per bun) exist only in frontend code today. Any wrong call to a Context method — or a future mobile client — bypasses them completely.

Rule: true business invariants live in the database and the server service layer. The UI may mirror them for fast feedback, but it is never the enforcement point.

3. Parallel near-duplicate models signal a missing domain decision

`Supplier` vs `Vendor`, and `Purchase` vs `PurchaseOrder`, are nearly identical shapes maintained as separate parallel systems. This usually means two workflows were built independently by different people.

Rule: resolve this *before* writing the Prisma schema. A database schema fossilizes ambiguity permanently; a prototype doesn't.

4. Client-side ID generation will eventually collide

`PO-2026-001`, `MIR-2026-001`, `EMP-001` and saree IDs are invented in browser memory. This works with one user in a demo and fails silently the moment two staff members create records at the same time.

Rule: anything requiring global uniqueness or sequence (invoice numbers, batch IDs, saree IDs) needs one source of truth — a DB sequence or a locked counter table.

5. UI completeness and security completeness are unrelated axes

Login, role selection, and admin-viewing-as-staff impersonation all exist as polished UI flows — and none of them verify identity. Auth is a `localStorage` flag. It is easy to mistake a finished login *screen* for a working auth *system*.

Rule: treat every client-side role check as a UX convenience only. Authorization is re-checked server-side on every request, without exception.

6. Mirror the frontend and backend vocabularies 1:1

When `features/qc/` on the frontend maps to `modules/qc/` on the backend, any engineer can trace a behavior across the whole stack without a map. This consistency is worth more long-term than either structure being individually clever.

Part 2 — Refactor Roadmap

Nine phases. Each ends with the app fully runnable — no phase leaves the codebase in a broken intermediate state.

#	Phase	Definition of done
0	Baseline & guardrails	ESLint + Prettier + strict tsconfig enforced; CI runs typecheck/lint/build on every push; a file-size lint warning above ~400 lines
1	Domain decisions	Written decision on Supplier vs Vendor and Purchase vs PurchaseOrder; canonical saree lifecycle state machine agreed; ID formats locked
2	Extract pure logic	Excel parsing/matching, QC deduction math, jari conversion, ID formatting moved into pure, unit-tested util modules; zero behavior change
3	Feature folders	Each domain moved into features/{name}/ with components/, hooks.ts, api.ts, types.ts; no file above ~400 lines
4	API seam	features/{name}/api.ts + React Query hooks exist and are what components call; api.ts still backed by Context internally
5	Backend scaffold	NestJS project running; Prisma schema migrated; docker-compose (Postgres/Redis/MinIO) up; health endpoint green
6	Backend modules	Domain modules implemented per the API contract, with server-side enforcement of QC/jari/ID/state-machine rules
7	Real auth	Phone+OTP, JWT access/refresh, server-side RBAC guards; localStorage auth deleted from the frontend
8	Cutover	api.ts swapped from Context-backed to HTTP-backed feature by feature; every *Context.tsx deleted; seed/mock arrays removed

Suggested order of attack within Phases 2–4: payments → weavers → inventory → finishing → everything else. The first two are the largest files and will validate the pattern under the hardest conditions.

Part 3 — Prompt Pack

Hand these to a coding agent one at a time, in order. Each is written to be self-contained. Review and commit between prompts.

PROMPT 0 — Baseline & guardrails

```
Set up code-quality guardrails for this React + TypeScript + Vite project before any refactor
begins. Do not change any application logic.

1. Enable TypeScript strict mode in tsconfig (strict, noUncheckedIndexedAccess,
   noImplicitOverride). Report how many type errors this surfaces; fix only the trivial
   ones and list the rest as follow-up work.
2. Add ESLint (typescript-eslint, react-hooks, import ordering) and Prettier with a shared
   config. Add "lint", "format", and "typecheck" npm scripts.
3. Add an ESLint rule (max-lines, warn at 400) so oversized files are visible.
4. Add a GitHub Actions workflow running typecheck, lint, and build on push and PR.
5. Produce a report listing every source file over 400 lines with its line count, sorted
   descending. Do not refactor them yet.

Leave application behavior byte-for-byte identical.
```

PROMPT 1 — Domain decisions (analysis only, no code changes)

```
Analyze this codebase and produce a written domain-decision document. Make no code changes.

1. Compare the Supplier model/flows against the Vendor model/flows. List every field and
   behavior that genuinely differs. Recommend whether these should be one entity with a
   type discriminator, or remain two entities, and justify it from actual business usage
   in the code.
2. Do the same for Purchase versus PurchaseOrder.
3. Trace the full saree lifecycle across the code (batch row creation, QC, stock,
   sale confirmation, finishing, dispatch, invoice). Document the states that actually
   exist in code today, and contrast them with the intended flow described in
   src/imports/pasted_text/erp-changelog.md. Propose one canonical state machine with
   explicit allowed transitions.
4. Catalogue every generated ID format in the codebase (saree, batch, PO, MIR, order,
   employee) with the exact rule and where it is generated.

Output as a markdown file at docs/domain-decisions.md, flagging any question that needs
a business-owner answer rather than an engineering one.
```

PROMPT 2 — Extract pure business logic

```
Extract pure business logic out of the oversized page components into tested utility
modules. Behavior must not change.

Targets, in order:
1. src/app/components/PaymentsPage.tsx  - Excel parsing and the weaver/vendor payment
   matching algorithms.
2. src/app/components/WeaversPage.tsx   - the weaver Excel import row parsing/matching.
3. QC pay-deduction math (defective = zero payable; semi = deduction capped at making
   charge; passed = full making charge).
4. Jari unit conversion (230 g per reel, 4 reels per bun) wherever it appears.
5. All ID formatting/parsing helpers.

For each: move the logic into a pure function module with no React imports, under the
relevant feature's utils/ folder. Add Vitest unit tests covering the real edge cases
(unmatched rows, malformed spreadsheet cells, deduction exceeding making charge,
fractional reels). Have the original component import the extracted function.

Report the before/after line count of each component touched.
```

PROMPT 3 — Restructure into feature folders

```
Restructure the frontend into feature folders. Do this ONE feature at a time, keeping the
app runnable and committing after each.

Target layout per domain:
  src/features/{name}/
    components/   presentational + container components, none over ~400 lines
    hooks.ts      React hooks (state, effects, data access)
    api.ts        data-access functions (still Context-backed for now)
    types.ts      the domain's TypeScript interfaces
    utils/        pure logic extracted in the previous phase
```

```
Start with: payments, then weavers, then inventory, then finishing.

Rules:
- Split any component over ~400 lines into a thin container plus focused child components.
- Move interfaces currently declared inside *Context.tsx into the feature's types.ts and
  re-export so nothing breaks mid-migration.
- Genuinely shared components (DateFilterBar, Pagination, SectionNavigator) go to
  src/shared/components/.
- Do not delete any *Context.tsx yet.
- Verify the dev server runs and the affected pages render after each feature.
```

PROMPT 4 — Introduce the API seam

```
Introduce a data-access seam so components stop calling React Context mutators directly.

1. Install and configure React Query (TanStack Query) with a QueryClientProvider at the app
   root and a centralized query-key factory at src/api/queryKeys.ts.
2. Create src/api/client.ts: a fetch/axios wrapper with base URL from env, an auth-header
   interceptor, and 401 handling. It is unused for now but must exist.
3. For each feature, implement features/{name}/api.ts exposing async functions named after
   the real operations (listBulkOrders, createBatch, approvePurchaseOrder, recordQcResult...).
   INTERNALLY these still read/write the existing React Context - the signatures and async
   shape are what matter.
4. Wrap each in a React Query hook in features/{name}/hooks.ts (useBulkOrders,
   useCreateBatch, ...).
5. Update components to consume ONLY these hooks. No component should import a Context
   mutator directly afterwards.

The point is the seam: after this phase, swapping api.ts to real HTTP must require no
component changes. Verify every affected page still works.
```

PROMPT 5 — Scaffold the backend

```
Scaffold the NestJS backend in a new backend/ directory at the repo root.

1. nest new backend (npm, TypeScript).
2. Install: @prisma/client, prisma, @nestjs/jwt, @nestjs/passport, passport, passport-jwt,
   @nestjs/bullmq, bullmq, ioredis, class-validator, class-transformer,
   @aws-sdk/client-s3, @aws-sdk/s3-request-presigner.
3. Create the folder structure: src/config, src/common/{decorators,guards,interceptors,
   filters,pipes,utils}, src/prisma, src/modules, src/jobs.
4. Add docker-compose.yml with postgres, redis, and minio, plus .env.example with every
   required variable, and env validation at startup (zod or joi) that fails fast.
5. Wire global ValidationPipe, a global HttpExceptionFilter, and a PrismaService.
6. Add a /health endpoint checking database and redis connectivity.
7. Write the Prisma schema from the entity list in the Backend Architecture Design document
   (users, weavers, factory_looms, design_library, saree_type_rates, batches,
   batch_saree_rows, material_issue_records + items, qc_records, finishing_assignments,
   finishing_returns, finishing_staff, quotations + quotation_sarees, bulk_orders,
   dispatch_records, inventory_records, sarees, sales, suppliers, vendors, purchases,
   purchase_orders, purchase_requests, firms, firm_financial_entries, weaver_payments,
   invoices, invoice_payments, vendor_payments, customers, audit_log, notifications).
   Include the foreign keys and enums described there. Add CHECK constraints so exactly one
   of weaver_id / factory_loom_id is set on saree-bearing tables.
8. Run the initial migration and confirm docker compose up plus npm run start:dev works.

Do not implement domain endpoints yet.
```

PROMPT 6 — Implement backend domain modules

```
Implement the backend domain modules against the API contract in the Backend Architecture
Design document. Build them in this order, one module per commit:

  users -> vendors/suppliers -> purchase-orders -> purchase-requests -> design-library ->
  batches -> factory-looms -> material-issues -> qc -> finishing -> quotations ->
  bulk-orders -> customers -> inventory -> sarees -> dispatch -> weaver-payments ->
  vendor-payments -> invoices -> firms -> rates-pricing -> notifications -> audit-log ->
  reports -> uploads

Every module follows the same shape: {name}.module.ts, {name}.controller.ts (routing and
guards only, no business logic), {name}.service.ts (business logic, Prisma access), and
dto/ with class-validator request/response classes.

Enforce these rules SERVER-SIDE, not just in the UI:
- QC pay rule: defective => payable 0; semi => deduction capped at making charge;
  passed => full making charge.
- Exactly one production source per saree (weaver loom OR factory loom).
```

```
- Saree lifecycle transitions restricted to the state machine from docs/domain-decisions.md.
- All sequential IDs generated by the database, never by the caller.
- Jari stored canonically in grams, exposed in both reels and buns.

Add an AuditLogInterceptor recording actor, action, and entity on every mutating request,
and unit tests per service covering the rules above.
```

PROMPT 7 — Real authentication and RBAC

```
Replace the prototype auth with a real implementation.

Backend:
1. auth module with POST /auth/otp/request and POST /auth/otp/verify (SMS provider behind
   an interface so it can be stubbed in dev), rate-limited per phone and per IP.
2. JWT access tokens (~15 min) plus refresh tokens in httpOnly cookies, with
   POST /auth/refresh and POST /auth/logout. Logout writes an audit-log event.
3. JwtAuthGuard, RolesGuard (admin, superadmin, worker, weaver, shop, accountant), and
   AccessLevelGuard for the finer-grained permissions the frontend already models.
4. A serializer/interceptor that strips monetary fields for money-restricted access levels,
   matching the existing MoneyAccess and DownloadAccess gates.
5. Admin/superadmin portal impersonation as an explicitly scoped, time-limited, audited
   token claim - never a client-side flag.

Frontend:
6. Rewrite AuthContext to hold tokens in memory with refresh-on-401 via src/api/client.ts.
   Delete the localStorage keys bk_auth_state and bk_original_admin_role.
7. Keep the existing per-role layout guards, but treat them as UX only - every request is
   re-authorized server-side.

Add e2e tests: expired token, wrong role, restricted access level, and OTP rate limiting.
```

PROMPT 8 — Cutover to the real backend

```
Cut the frontend over from Context-backed data to the real API, one feature at a time.

For each feature, in this order (payments, weavers, inventory, finishing, then the rest):
1. Rewrite features/{name}/api.ts so its functions call the real endpoints through
   src/api/client.ts instead of React Context. Signatures must not change.
2. Verify the feature's pages work end to end against the running backend.
3. Delete the corresponding *Context.tsx and its hardcoded seed arrays.
4. Commit.

Then finish the job:
5. Delete the pseudo-random dataset generator in SalesContext and replace it with real
   aggregation endpoints.
6. Remove every remaining mock/seed array from src/.
7. Confirm no component imports a Context mutator, and that no business rule remains
   enforced solely on the client.

Report any place where frontend behavior silently depended on mock data shapes that the
real API does not reproduce.
```

Part 4 — Standards to Hold the Line

Guardrails that keep the codebase professional after the refactor lands.

4.1 Code conventions

Area	Standard
File size	Components under ~400 lines; services under ~300. Over that, split by responsibility.
Naming	Frontend features/{domain} and backend modules/{domain} use identical domain names.
Components	Container components fetch (hooks); presentational components take props and render. Never mix.
Business logic	Pure functions in utils/, no React or Prisma imports, always unit-tested.
Types	Backend DTOs are the source of truth; frontend types mirror them. No divergent duplicates.
Controllers	Routing, guards, and validation only. Zero business logic.
Money	Store as integer minor units or decimal — never floating point.
Dates	Store UTC timestamps; format at the presentation layer only.

4.2 Pull-request checklist

- No new file over 400 lines, and no business rule added that exists only on the client.
- Every new endpoint has a role guard and a DTO with validation.
- Every new business rule has a unit test, including its failure case.
- No hardcoded IDs, mock arrays, or seed data added to application code.
- Frontend and backend naming for the same concept match exactly.
- Typecheck, lint, and build pass in CI.

4.3 Testing expectations

- **Unit** — every pure util (Excel matching, QC deduction, jari conversion, ID formatting) and every service business rule.
- **Integration** — each module's endpoints against a real test database, including RBAC rejection paths.
- **E2E** — the critical flows: batch creation → material issue → QC → finishing → dispatch → invoice → payment.
- Not required: exhaustive UI snapshot tests. Prioritize logic and authorization over pixels.

4.4 What 'done' looks like

The refactor is complete when: no `*Context.tsx` holds business data; every business invariant is enforced server-side and covered by a test; auth is real on both sides; no sequential ID is generated in a browser; and the frontend and backend folder trees mirror each other domain for domain.

Run one prompt per session, review the diff, and commit before moving on. The phases are ordered so that stopping after any one of them still leaves a codebase in better shape than before it started.